

Анализ структуры сетей и оценка расстояний методом landmark-вершин

Команда Team_01 • Python

Дискретная математика • 2025–2026

Задача 1: анализ структуры сети

Граф $G = (V, E)$ — модель сети. Вычисляем метрики, характеризующие структуру.

Плотность: $\rho = \frac{2|E|}{|V|(|V|-1)}$ — доля существующих рёбер

Степени вершин: $\deg(v)$ — число соседей; min/max/mean; распределение

WCC (слабые компоненты): BFS от непосещённой вершины → вся компонента.

Повторяем, пока есть непосещённые. $O(|V| + |E|)$

SCC (сильные компоненты) — алгоритм Косарайю:

1. DFS с записью порядка выхода (post-order)
2. На обращённом графе DFS в порядке убывания → каждая DFS = одна SCC

$O(|V| + |E|)$, 2 прохода

Задача 1: кластеризация и диаметр

Коэффициент кластеризации

Мера «сообщества» — насколько соседи вершины связаны между собой:

$$C(v) = \frac{\text{число пар соседей } v, \text{ между которыми есть ребро}}{\binom{\deg(v)}{2}}$$

- Локальный: для каждой вершины
- Средний: $\bar{C} = \frac{1}{|V|} \sum_v C(v)$
- Глобальный: $\frac{3 \times \text{число треугольников}}{\text{число триплетов}}$

Подсчёт треугольников: итерация по вершинам, проверка рёбер между соседями с сортировкой по степени (Degree Ordering) — сложность $\sum_v \deg(v)^2$, на разреженных графах приемлемо.

Задача 1: оценка диаметра

Точный диаметр $D = \max_{u,v} d(u,v)$ — $O(|V|(|V| + |E|))$, неприменим для больших графов.

Double-sweep (2-аппроксимация):

1. BFS из случайной вершины $s \rightarrow$ находим самую дальнюю a
2. BFS из $a \rightarrow$ находим самую дальнюю b
3. $d(a,b)$ — нижняя граница диаметра, гарантированно $\geq D/2$

Random-pair sampling: выбор случайных пар (u,v) , BFS от u до v . Собираем перцентили (P50, P90, P100) для оценки распределения расстояний.

Snowball sampling: BFS-подграф фиксированного размера от случайного старта
 $\rightarrow$ те же метрики внутри локального куска графа.

Задача 1: робастность сети

Симулируем удаление узлов (с шагами 0%, 5%, ..., 50%) и измеряем долю largest weak component от исходного графа.

Random attack: на каждом шаге удаляем $p\%$ случайных вершин.

Показывает **общую устойчивость** сети к случайным сбоям.

Targeted attack: удаляем $p\%$ вершин с **наибольшей степенью** (хабы).

Показывает **зависимость сети от ключевых узлов**. Если граф резко распадается — значит, он держится на хабах.

На каждом шаге: $O(|V| + |E|)$ на BFS для largest component.

Что такое landmark-вершины? — идея

Точное расстояние $d(u, v)$ требует BFS от u до v — $O(|V| + |E|)$ на запрос. Если запросов много (сотни тысяч пар), это неприемлемо.

Landmark-подход: выбираем небольшое множество «ориентирных» вершин $L = \{\ell_1, \dots, \ell_k\}$, предвычисляем расстояния от них до **всех** вершин графа.

Preprocessing (один раз): k BFS-ов = $O(k \cdot (|V| + |E|))$

Запрос (много раз): $O(k)$ — просто суммируем числа

Что такое landmark-вершины? — почему работает?

Если мы знаем $d(\ell, u)$ и $d(\ell, v)$ для каждого ориентира ℓ , то по **неравенству треугольника**:

$$d(u, v) \leq d(u, \ell) + d(\ell, v)$$

Это **верхняя граница**. Берём минимум по всем ориентирам — получаем наилучшую верхнюю оценку:

$$d(u, v) \approx \min_{\ell \in L} (d(u, \ell) + d(\ell, v))$$

Как выбирать landmark? — random / highest-degree

Качество оценки зависит от покрытия графа ориентирами. Чем равномернее — тем точнее.

Random: k случайных вершин из largest weak component. $O(1)$. Минус: при малом k ориентиры могут сгруппироваться.

Highest-degree: k вершин с максимальной степенью. Хабы «видят» большую часть графа. $O(|V| \log |V|)$ на сортировку.

Как выбирать landmark? — coverage

Жадная аппроксимация **k -центра**: минимизируем макс. расстояние от любой вершины до ближайшего ориентира.

1. Первый ориентир — вершина с макс. степенью
2. BFS от неё → расстояния до всех вершин
3. На каждом шаге выбираем вершину, **максимально удалённую** от уже выбранных (max из min-расстояний)
4. BFS от нового ориентира, обновляем min-расстояния

Сложность: $O(k \cdot (|V| + |E|))$ — k BFS-ов. Дорого, но даёт лучшее покрытие.

Алгоритм Basic: preprocess

Preprocessing: для каждого ориентира $\ell \in L$ запускаем BFS от ℓ , запоминаем $dist[\ell][v]$ для всех v .

Храним: k векторов длины $|V|$ — $k \cdot |V|$ чисел (int). Для графа на 1М вершин и $k = 16$ это ~64MB — приемлемо.

```
dist[ $\ell_1$ ] = [d( $\ell_1, v_1$ ), d( $\ell_1, v_2$ ), ..., d( $\ell_1, v_n$ )]  
dist[ $\ell_2$ ] = [d( $\ell_2, v_1$ ), d( $\ell_2, v_2$ ), ..., d( $\ell_2, v_n$ )]  
...  
dist[ $\ell_k$ ] = [d( $\ell_k, v_1$ ), d( $\ell_k, v_2$ ), ..., d( $\ell_k, v_n$ )]
```

Алгоритм Basic: оценка (query)

Для запроса (u, v) :

$$d(u, v) \approx \min_{\ell \in L} (d(\ell, u) + d(\ell, v))$$

1. Для каждого ориентира ℓ : берём $d(\ell, u)$ и $d(\ell, v)$ из предвычисленного вектора
2. Складываем: $candidate = dist[\ell][u] + dist[\ell][v]$
3. Берём минимум по всем ℓ

Почему это верхняя граница? По неравенству треугольника:

$$d(u, v) \leq d(u, \ell) + d(\ell, v)$$

Минимум по всем ориентирам — наилучшая оценка.

Сложность запроса: $O(k)$ — k сложений и k сравнений. Для $k = 16$ это ~ 40 операций. Точный BFS потребовал бы $O(|V| + |E|)$ — миллионы операций.

Что такое LCA и зачем он нужен?

LCA (Least Common Ancestor) — наименьший общий предок двух вершин в дереве.

Как строим дерево?

Каждый BFS от ориентира ℓ порождает **дерево кратчайших путей (SP-tree)**: для каждой вершины v запоминаем родителя $p(v)$ — предыдущую вершину на кратчайшем пути от ℓ к v .

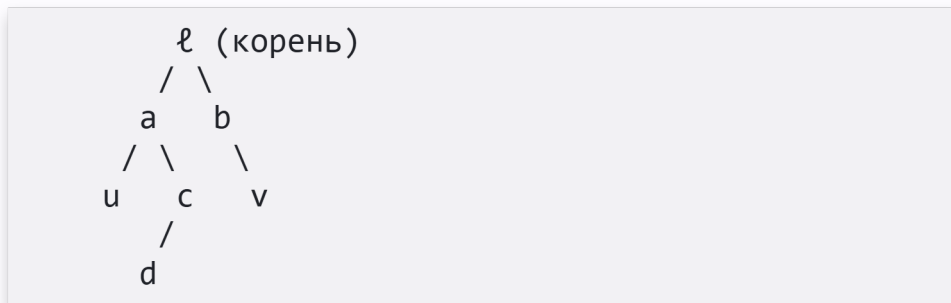
Свойство

В SP-дерево кратчайший путь от ℓ к v — единственный. Более того, для любых u, v

$$d_{SP}(u, v) = \overset{\vdots}{depth(u) + depth(v) - 2 \cdot depth(LCA(u, v))}$$

где $depth$ — глубина в SP-дерево от корня ℓ .

LCA: пример



$$LCA(u, v) = \ell, \quad d_{SP}(u, v) = 2 + 2 - 2 \cdot 0 = 4$$

$$LCA(u, d) = a, \quad d_{SP}(u, d) = 2 + 3 - 2 \cdot 1 = 3$$

Что даёт LCA?

$d_{SP}(u, v)$ — **точное** расстояние в дереве. В исходном графе между u и v может быть более короткий путь (не через ℓ). Но $d_{SP}(u, v) \geq d(u, v)$, и чем «лучше» ориентир, тем ближе оценка к истине.

Binary lifting — предвычисление

Binary lifting (метод двоичного подъёма): для каждой вершины v храним $jump[v][p]$ — предок на 2^p уровней вверх. $O(\log n)$ памяти на вершину.

Как строим?

$jump[v][0] = parent(v)$ — непосредственный родитель

$jump[v][1] = jump[jump[v][0]][0]$ — предок через 2 уровня (родитель родителя)

$jump[v][2] = jump[jump[v][1]][1]$ — предок через 4 уровня

В общем виде: $jump[v][p] = jump[jump[v][p-1]][p-1]$

Для n вершин нужно $\lceil \log_2 n \rceil$ степени двоек. Для $n = 10^6$ — 20 уровней.

Binary lifting — запрос LCA

Шаг 1: выравнивание глубин

Пусть $depth[u] \geq depth[v]$. Нужно поднять u так, чтобы $depth[u] = depth[v]$.
Разбиваем $delta = depth[u] - depth[v]$ на сумму степеней двоек. Для каждого бита p , где $delta$ имеет 1: $u = jump[u][p]$.

Шаг 2: поиск LCA

Если $u = v$ — LCA найден.

Иначе поднимаем обе вершины **синхронно**: для p от $\log n$ до 0:

- Если $jump[u][p] \neq jump[v][p]$ — поднимаем обе

После цикла $parent(u) = parent(v) = LCA$.

Сложность: $O(\log n)$ — ровно 2 прохода по битам.

Алгоритм BFS: preprocessing

SP-дерево (shortest-path tree): BFS от ориентира ℓ фиксирует для каждой вершины v предшественника $p(v)$ — предыдущую вершину на кратчайшем пути от ℓ к v .

ℓ (корень): $p(\ell) = \text{None}$
её соседи: $p(\text{сосед}) = \ell$
их соседи: $p(\dots) = \text{предыдущая вершина}$

Храним для каждого ориентира ℓ :

- $dist[\ell][v]$ — расстояние $d(\ell, v)$
- $parents[\ell][v]$ — $p(v)$ для binary lifting
- $depths[\ell][v]$ — глубина в SP-дереве

Binary lifting: $jump_{\ell}[v][p]$ — предок на 2^p уровней.

Алгоритм BFS: оценка и комбинация

Оценка через LCA

Для запроса (u, v) по каждому ориентиру ℓ :

1. $lca = LCA_\ell(u, v)$ — находим через binary lifting за $O(\log n)$
2. Оценка расстояния **в дереве**:

$$d_{SP}(u, v) = depths_\ell[u] + depths_\ell[v] - 2 \cdot depths_\ell[lca]$$

3. $d(u, v) \approx \min_{\ell \in L} d_{SP}(u, v)$

Комбинация с Basic

В итоговой функции `estimate()` вычисляем **обе** оценки и берём минимум:

$$d(u, v) \approx \min(Basic(u, v), BFS(u, v))$$

Basic использует сумму расстояний $(d(\ell, u) + d(\ell, v))$. BFS использует LCA-формулу $(depth(u) + depth(v) - 2 \cdot depth(LCA))$. Две разные верхние границы — минимум всегда не хуже каждой по отдельности.

Как работает `estimate()` — итог

В коде (`distance_estimation.py`):

```
def estimate(source, target):
    candidates = []
    # Basic:  $d(l, u) + d(l, v)$  — всегда верхняя граница
    candidates.append(estimate_basic(source, target))
    # BFS: tree_distance через LCA — обычно точнее
    candidates.append(estimate_bfs(source, target))
    return min(c for c in candidates if c is not None)
```

Почему Basic всё ещё нужен?

$d_{SP}(u, v)$ — расстояние **в дереве**, оно может быть **больше** истинного $d(u, v)$, если кратчайший путь не проходит через корень ℓ . Basic-оценка через сумму расстояний — другая верхняя граница. Минимум двух подходов даёт лучшую оценку.

Структура проекта — обзор

```
discrete-math-MM-2025-2026/
├── src/network_analysis/
│   ├── graph.py
│   ├── io_utils.py
│   ├── datasets.py
│   ├── metrics/
│   │   ├── basic.py
│   │   ├── clustering.py
│   │   ├── distance.py
│   │   └── robustness.py
│   ├── distance_estimation.py
│   ├── cli.py / distance_cli.py
│   ├── report.py
│   ├── plotting.py
│   ├── demo_cli.py
│   └── comparison.py
├── run_analysis.py
├── run_distance_estimation.py
├── run_all_datasets.py
├── run_compare.py
├── run_demo.py
└── results/
```

исходный код (основной пакет)
графовое ядро
загрузка данных
реестр датасетов
метрики (подпакет)
базовые метрики
кластеризация
расстояния и диаметр
атаки на сеть
landmark index (основной)
CLI entry points
сбор отчёта
графики
интерактивное демо
сравнение датасетов
скрипт запуска (задача 1)
скрипт запуска (задача 2)
батч-запуск по всем датасетам
сравнение результатов
запуск демо
выходные данные

graph.py — графовое ядро

```
@dataclass
class Graph:
    directed: bool
    out_adj: Dict[str, Set[str]] # source → {targets}
    in_adj: Dict[str, Set[str]] # target → {sources}
```

Методы:

- `add_node / add_edge` — наполнение графа
- `node_count / edge_count` — число вершин/рёбер
- `neighbors / undirected_neighbors` — получение соседей (с учётом направленности)
- `degree` — степень вершины
- `induced_subgraph` — подграф на заданном множестве вершин
- `remove_nodes` — удаление вершин (через `induced_subgraph`)
- `reversed` — обращённый граф (для SCC)
- `edge_list` — список рёбер

Хранение: `Dict[str, Set[str]]` — гибко (строки-ключи), но памятьёмко.

io_utils.py и datasets.py — загрузка данных

io_utils.py

Парсинг графов из файлов трёх форматов:

- **TXT (edge list):** `source target` на каждой строке. Пустые строки и комментарии игнорируются. Если в шапке есть `# Nodes: N Edges: M` — `size_header_edge_list`.
- **CSV:** колонки разделены запятыми/табами. Первая строка может быть заголовком (`source,target`).
- **MTX (Matrix Market):** формат разреженных матриц. Строки после `%%` — данные.

datasets.py

Реестр 12 датасетов: имя файла, формат, directed-флаг, включение в distance-эксперименты.

metrics/basic.py — базовые метрики

Функции для первичного анализа графа:

- `density(graph)` — $\rho = \frac{|E|}{|V|(|V|-1)/2}$ (для неориент.) или $\frac{|E|}{|V|(|V|-1)}$ (для ориент.)
- `weakly_connected_components(graph)` — BFS из каждой непосещённой вершины. Возвращает список компонент. $O(|V| + |E|)$
- `strongly_connected_components(graph)` — алгоритм Косарайю: прямой DFS $\rightarrow$ post-order, затем обратный DFS в этом порядке. Для неориентированных графов — WCC.
- `degree_stats(graph)` — min/max/mean степени
- `degree_distribution(graph)` — Dict {степень: доля вершин}
- `largest_component_share(components, total)` — доля вершин в крупнейшей компоненте

metrics/distance.py — расстояния и диаметр

- `bfs_distances(graph, source)` — BFS от `source`, возвращает Dict {вершина: расстояние}
- `eccentricity(graph, node)` — макс. расстояние от `node` до любой вершины (через BFS)
- `double_sweep_diameter(graph, nodes)` — 2-аппроксимация диаметра: BFS от `random start` → `farthest` → BFS от `farthest`
- `exact_diameter(graph, nodes)` — точный диаметр через all-pairs BFS. Ограничен 2000 вершин (для больших графов — `double-sweep`)
- `sample_pair_distances(graph, nodes, sample_size)` — случайные пары, BFS до `target` для каждой. Возвращает список расстояний.
- `snowball_sample(graph, seeds, target_size)` — BFS-подграф фикс. размера.
- `summarize_distances(distances)` — P50, P90, P100 из списка расстояний.

metrics/clustering.py — кластеризация

Алгоритмы для измерения «сообществ» в графе:

- `triangle_count(graph)` — подсчёт треугольников через Degree Ordering: сортируем вершины по убыванию степени, для каждой проверяем рёбра между соседями с меньшей степенью. Сложность $\sum_v \deg(v)^2/2$.
- `local_clustering_coefficients(graph)` — для каждой вершины: $C(v) = \frac{\text{ребра между её соседями}}{\binom{\deg(v)}{2}}$
- `average_clustering_coefficient(graph)` — среднее арифметическое локальных коэффициентов по всем вершинам.
- `global_clustering_coefficient(graph)` — $\frac{3 \times \text{треугольники}}{\text{триплеты}}$ (число связных троек)

metrics/robustness.py — атаки на сеть

Общий подход: на каждом шаге удаляем $p\%$ узлов, запускаем BFS для largest weak component, записываем долю.

- `random_attack(graph, steps, seed)` — на каждом проценте `steps` удаляем случайные вершины. Результат — список Dict с `removed_percent` и `largest_weak_component_share`.
- `targeted_attack(graph, steps)` — вершины удаляются в порядке убывания степени (от самых «важных» к наименее). Результат аналогичный.
- `largest_weak_component_fraction(graph)` — BFS от случайного старта, находим компоненту через `bfs_ignore` (игнорирование удалённых вершин).

По 9 шагов: 0%, 5%, 10%, 15%, 20%, 25%, 30%, 40%, 50% удалённых вершин.

distance_estimation.py — landmark index (главный файл)

Основные функции (≈ 400 строк):

Функция	Назначение
<code>select_landmarks()</code>	Выбор ориентиров (random / highest-degree / coverage)
<code>bfs_distances_with_parents()</code>	BFS с записью родителей (SP-дерево)
<code>_build_jump_table()</code>	Binary lifting: $jump[v][p]$ для LCA
<code>_lca()</code>	LCA-запрос за $O(\log n)$
<code>LandmarkTree</code>	Dataclass: root, distances, parents, depths, jump
<code>LandmarkIndex</code>	Dataclass: список LandmarkTree + алгоритм
<code>estimate_basic()</code>	$d(u, v) \approx \min_{\ell} (d(\ell, u) + d(\ell, v))$
<code>estimate_bfs()</code>	$d(u, v) \approx \min_{\ell} \text{tree_distance}_{\ell}(u, v)$
<code>estimate()</code>	min из basic + bfs
<code>evaluate_distance_estimator()</code>	Сравнение с точным BFS на N парах
<code>save_distance_experiment()</code>	Сохранение результатов

report.py и plotting.py — вывод результатов

`build_report(graph, output_dir, ...)` — оркестратор:

1. WCC и SCC, берёт largest weak component
2. Degree distribution
3. Local clustering на largest weak
4. Double-sweep diameter на largest weak
5. Random-pair + snowball distance sampling
6. Double-sweep при разных выборках
7. Exact diameter (≤ 2000 вершин)
8. Random + targeted attack (CSV)
9. Сохраняет `summary.json`, degree distribution .png

Runner-скрипты — автоматизация

Скрипты в корне проекта — тонкие обёртки над CLI:

<code>run_analysis.py</code>	<code>→ cli.main()</code>	<code># 1 датасет</code>
<code>run_distance_estimation.py</code>	<code>→ distance_cli.main()</code>	<code># 1 датасет</code>
<code>run_all_datasets.py</code>	<code>→ цикл по datasets.py</code>	<code># все 12 датасетов</code>
<code>run_distance_batch.py</code>	<code>→ distance_batch.main()</code>	<code># батч landmark-экспериментов</code>
<code>run_compare.py</code>	<code>→ comparison.main()</code>	<code># сравнение summary.json</code>
<code>run_demo.py</code>	<code>→ demo_cli.main()</code>	<code># интерактив</code>

Особенности `run_all_datasets.py`:

- Для каждого из 12 датасетов загружает граф, запускает `build_report()`
- Результаты: `results/<dataset_name>/summary.json`
- Данные сохраняются для последующего сравнения через `run_compare.py`

Запуск

```
# Полный анализ одного датасета
python run_analysis.py datasets\CA-GrQc.txt --output-dir results\CA-GrQc

# Landmark distance estimation
python run_distance_estimation.py datasets\CA-GrQc.txt --output-dir results\distance_test

# Все 12 датасетов последовательно
python run_all_datasets.py

# Сравнение метрик датасетов
python run_compare.py

# Интерактивный демо-режим
python run_demo.py datasets\CA-GrQc.txt summary
```

Зависимости: только standard library. matplotlib — опционально.

Результаты — метрики графов (1/2)

Датасет	N	E	ρ	WCC	deg	Δ	C	D
CA-GrQc	5 241	14 484	.00106	354	5.5	48 260	.530	15
CA-AstroPh	18 771	198 050	.00112	289	21.1	1 351 441	.631	14
soc-wiki-Vote (d)	889	2 914	.00369	1	6.6	2 119	.153	13
Wiki-Vote (d)	7 115	103 689	.00205	24	28.3	608 389	.141	7
musae_git_edges	37 700	289 003	.00041	1	15.3	523 810	.168	11
Email-EuAll	265 009	364 481	.00001	15 631	2.8	267 313	.067	14

N — вершины, **E** — рёбра, **ρ** — плотность, **WCC** — слабые компоненты, **deg** — ср. степень, **Δ** — треугольники, **C** — коэфф. кластеризации, **D** — диаметр (double-sweep)

Результаты — метрики графов (2/2)

Датасет	N	E	ρ	WCC	deg	Δ	C	D
web-Stanford (d)	281 903	2 312 497	.00003	365	14.1	11 329 473	.598	164
web-NotreDame (d)	325 729	1 469 679	.00001	1	6.7	8 910 005	.235	46
ca-coauthors-dblp	540 486	15 245 729	.00010	1	56.4	444 095 058	.802	23
web-Google (d)	875 713	5 105 039	.00001	2 746	9.9	13 391 903	.514	24
com-youtube.ungraph	1 134 890	2 987 624	.000005	1	5.3	3 056 386	.081	24
vk	3 215 720	17 414 510	.000003	24 337	10.8	108 030 337	.049	19

N — вершины, **E** — рёбра, **ρ** — плотность, **WCC** — слабые компоненты, **deg** — ср. степень, **Δ** — треугольники, **C** — коэфф. кластеризации, **D** — диаметр (double-sweep)

**Жирным выделены рекордные значения среди всех 12 датасетов*

Результаты — random attack

50% nodes removed. Largest weak component share:

Высокая устойчивость	Средняя	Низкая
ca-coauthors-dblp 0.930	web-Google 0.719	CA-GrQc 0.494
musae_git_edges 0.884	web-Stanford 0.633	web-NotreDame 0.452
CA-AstroPh 0.849	com-youtube 0.594	Email-EuAll 0.435
soc-wiki-Vote 0.813 · vk 0.805 · Wiki-Vote 0.800		

“ 70% сетей сохраняют largest component — случайные отказы не критичны. ”

Результаты — targeted attack

50% highest-degree nodes removed. Largest weak component share:

Держится	Разрушен полностью
ca-coauthors-dblp 0.749	soc-wiki-Vote 0.013 · CA-AstroPh 0.008
	web-Stanford 0.005 · Google ≈ 0.00008
	Email ≈ 0.00002 · vk ≈ 0.00001 · youtube $\approx$ 0.000002

Почти все сети **полностью разрушаются** при удалении хабов. ca-coauthors-dblp — исключение.

Результаты — ключевые наблюдения

Метрики: ca-coauthors-dblp — макс. кластеризация (0.802). web-Stanford — макс. диаметр (164). vk — самый большой (3.2M), много компонент (24 337). Email-EuAll — мин. средняя степень (2.8).

Робастность: случайные атаки — ≥ 0.435 (хорошо). Целенаправленные — ≤ 0.013 (разрушены), кроме ca-coauthors-dblp (0.749). Зависимость от хабов — уязвимость scale-free сетей.

Результаты — landmark estimation

Настройки: basic + lca, $k \in \{8, 16\}$, random + highest_degree, 30 случайных пар на датасет. Лучшая точность и лучшая скорость (query time) среди всех 8 комбинаций.

Датасет	N комп.	Точность (alg/k/strat)	Совп.	MAE	Speedup	Скорость (alg/k/strat)	Speedup
CA-GrQc	4 158	lca/16/rnd	.833	0.23	58x	basic/8/hd	734x
soc-wiki-Vote	889	lca/16/rnd	.967	0.03	22x	basic/8/rnd	305x
Wiki-Vote	7 066	lca/16/rnd	.733	0.27	223x	basic/8/rnd	3 406x
CA-AstroPh	17 903	lca/16/hd	.500	0.50	556x	basic/8/rnd	8 111x
musae_git_edges	37 700	lca/16/hd	.867	0.13	1 110x	basic/8/rnd	13 831x
Email-EuAll	224 832	lca/16/hd	.700	0.30	2 784x	basic/8/hd	25 260x
web-NotreDame	325 729	lca/16/rnd	1.000	0.00	4 025x	basic/8/hd	61 480x

Лучшая точность: web-NotreDame (lca/16/random) — 100% exact match, MAE = 0.

Лучшая скорость: web-NotreDame (basic/8/hd) — 61 480x ускорение. LCA с $k=16$ точнее на всех датасетах. Basic с $k=8$ быстрее на запросе, но менее точен. На малых графах (soc-wiki-Vote) ускорение скромное (22–305x), на больших — до 61 480x.

Спасибо за внимание!